

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
print("Imported Successfully")
```

Imported Successfully

```
df = pd.read_csv('/content/car_price_prediction_.csv')
display(df.head())
```

|   | Car ID | Brand | Year | Engine Size | Fuel Type | Transmission | Mileage | Condition | Price    | Model    |  |
|---|--------|-------|------|-------------|-----------|--------------|---------|-----------|----------|----------|-------------------------------------------------------------------------------------|
| 0 | 1      | Tesla | 2016 | 2.3         | Petrol    | Manual       | 114832  | New       | 26613.92 | Model X  |                                                                                     |
| 1 | 2      | BMW   | 2018 | 4.4         | Electric  | Manual       | 143190  | Used      | 14679.61 | 5 Series |                                                                                     |
| 2 | 3      | Audi  | 2013 | 4.5         | Electric  | Manual       | 181601  | New       | 44402.61 | A4       |                                                                                     |
| 3 | 4      | Tesla | 2011 | 4.1         | Diesel    | Automatic    | 68682   | New       | 86374.33 | Model Y  |                                                                                     |
| 4 | 5      | Ford  | 2009 | 2.6         | Diesel    | Manual       | 223009  | Like New  | 73577.10 | Mustang  |                                                                                     |

```
# Check dataset information
print(df.info())
```

```
# Check for missing values
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
```

```
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Car ID           2500 non-null   int64
1   Brand            2500 non-null   object
2   Year             2500 non-null   int64
3   Engine Size      2500 non-null   float64
4   Fuel Type        2500 non-null   object
5   Transmission     2500 non-null   object
6   Mileage          2500 non-null   int64
7   Condition        2500 non-null   object
8   Price            2500 non-null   float64
9   Model            2500 non-null   object
dtypes: float64(2), int64(3), object(5)
memory usage: 195.4+ KB
None
Car ID           0
Brand            0
Year             0
Engine Size      0
Fuel Type        0
Transmission     0
Mileage          0
Condition        0
Price            0
Model            0
dtype: int64
```

```
# Select only numeric features
df_numeric = df.select_dtypes(include=[np.number])

# Drop rows with missing values
df_numeric = df_numeric.dropna()

# Separate features (X) and target variable (y)
X = df_numeric.drop(columns=[df_numeric.columns[-1]])
y = df_numeric[df_numeric.columns[-1]]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
#Linear Regression Model
model = LinearRegression()
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
#To make Predictions
y_pred = model.predict(X_test)
```

```
#Model Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print("Mean Squared Error (MSE):", mse)
print("R-squared (R2) Score:", r2)
```

```
Mean Squared Error (MSE): 758123609.8201208
R-squared (R2) Score: -0.0007527886313760668
```

```
#Visualizations
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Car Prices")
plt.ylabel("Predicted Car Prices")
plt.title("Actual vs Predicted Car Prices")
plt.show()
```



